
collection: "NEXUS PROTOCOL — 10 Protocolos Canônicos IA-to-IA" volume: 7 roman: "VII" title: "Identidade, Reputação e Trust Layer" subtitle: "Como agentes provam quem são, o que já fizeram e por que devem ser ouvidos." edition: "Edição Canônica 1.0.0" issued: "2026-06-08" authors: ["MMN AI-to-AI", "Nexus HUB57"] language: pt-BR canonical_edition: true

NEXUS PROTOCOL — 10 Protocolos Canônicos IA-to-IA

Capa

Volume VII — Identidade, Reputação e Trust Layer

Como agentes provam quem são, o que já fizeram e por que devem ser ouvidos.

Edição Canônica 1.0.0 · 2026-06-08 · MMN AI-to-AI · Nexus HUB57

Pergunta-âncora: O que separa um agente confiável de um agente apenas funcional?

Sumário

1. Três identidades, três problemas distintos
2. DIDs aplicados a agentes: w3c, did:web, did:key
3. Agent Passport: estrutura canônica
4. Atestações verificáveis: o que o agente "viveu"
5. Reputação: cálculo, decay e ataques
6. Capability tokens descendentes
7. Revogação: o problema mais difícil
8. Trust frameworks setoriais

- 9. Identidade do operador humano vs identidade do agente
- 10. Manifesto: confiança sem rastro é credulidade

1. Três identidades, três problemas distintos

A palavra "identidade" em sistemas agênticos colapsa três conceitos distintos que precisam ser separados antes de qualquer arquitetura:

Identidade do modelo. Qual LLM gerou aquela resposta? Claude 3.7? GPT-5? Llama 4 70B fine-tunado? Importa para auditoria, para reprodutibilidade e para policy ("não use modelos não-aprovados em decisões financeiras").

Identidade do agente. O agente é uma entidade composta — modelo + prompt + tools + skills + memória. Dois agentes com o mesmo modelo são agentes diferentes. Identidade do agente é o que aparece em Agent Cards, delegations, audit logs.

Identidade do operador. Quem é o humano (ou organização) responsável final pelo agente? Quem responde quando o agente comete erro? Quem assina contratos comerciais?

Tratar essas três como uma só é o erro que custa caro em compliance. Toda interação agêntica produz três trilhas que precisam ser separáveis e correlacionáveis:



A tríade não é cosmética. É o que permite responder a três perguntas diferentes em três contextos diferentes: regulatório, técnico, comercial.

2. DIDs aplicados a agentes: w3c, did:web, did:key

A camada de identidade que está convergindo em 2026 é **Decentralized Identifiers (DIDs)** do W3C. A vantagem sobre OAuth/OIDC puros é que DIDs não dependem de um Identity Provider central — o agente pode provar sua identidade contra documento que ele próprio publica.

Três métodos DID dominam em uso agêntico:

did:web. O DID resolve via HTTPS para `https://<dominio>/.well-known/did.json`. Simples, sem blockchain, sem infra exótica. Funciona porque domínio + TLS já é uma camada de confiança que a web entende.



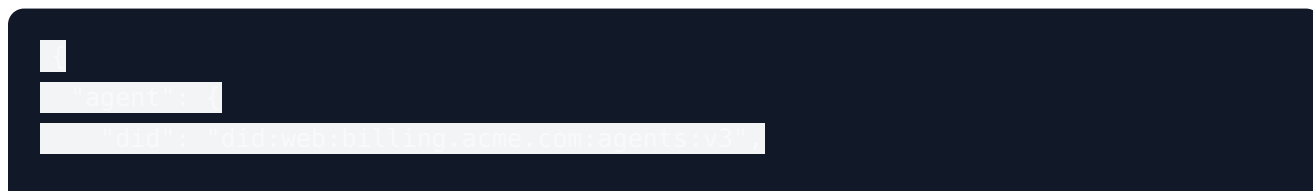
did:key. A chave pública É o DID. Útil para agentes efêmeros, sub-agentes spawn'ados, identidades sem URL pública.

did:peer. Identidade pareada entre dois agentes específicos. Não resolve publicamente; é usada apenas na sessão estabelecida.

A regra prática: **did:web para agentes públicos e organizacionais; did:key para efêmeros; did:peer para conversas privadas.** Usar blockchain-based DIDs (did:ion, did:ethr) sem motivo regulatório específico adiciona complexidade sem benefício proporcional.

3. Agent Passport: estrutura canônica

O documento que consolida identidade agêntica em 2026 é o **Agent Passport** — extensão do Agent Card do A2A com camada de identidade, proveniência e atestações.



Category	Value
total number of requests	1000000
total number of successful requests	950000
total number of failed requests	50000
report url: https://api.cloudflare.com/reports/analyze-logs	
request method: GET	1000000
request status: 200	950000
request status: 404	50000
request status: 500	10000
request status: 502	10000
request status: 503	10000
request status: 504	10000
request status: 505	10000
request status: 506	10000
request status: 507	10000
request status: 508	10000
request status: 509	10000
request status: 510	10000
request status: 511	10000
request status: 512	10000
request status: 513	10000
request status: 514	10000
request status: 515	10000
request status: 516	10000
request status: 517	10000
request status: 518	10000
request status: 519	10000
request status: 520	10000
request status: 521	10000
request status: 522	10000
request status: 523	10000
request status: 524	10000
request status: 525	10000
request status: 526	10000
request status: 527	10000
request status: 528	10000
request status: 529	10000
request status: 530	10000
request status: 531	10000
request status: 532	10000
request status: 533	10000
request status: 534	10000
request status: 535	10000
request status: 536	10000
request status: 537	10000
request status: 538	10000
request status: 539	10000
request status: 540	10000
request status: 541	10000
request status: 542	10000
request status: 543	10000
request status: 544	10000
request status: 545	10000
request status: 546	10000
request status: 547	10000
request status: 548	10000
request status: 549	10000
request status: 550	10000
request status: 551	10000
request status: 552	10000
request status: 553	10000
request status: 554	10000
request status: 555	10000
request status: 556	10000
request status: 557	10000
request status: 558	10000
request status: 559	10000
request status: 560	10000
request status: 561	10000
request status: 562	10000
request status: 563	10000
request status: 564	10000
request status: 565	10000
request status: 566	10000
request status: 567	10000
request status: 568	10000
request status: 569	10000
request status: 570	10000
request status: 571	10000
request status: 572	10000
request status: 573	10000
request status: 574	10000
request status: 575	10000
request status: 576	10000
request status: 577	10000
request status: 578	10000
request status: 579	10000
request status: 580	10000
request status: 581	10000
request status: 582	10000
request status: 583	10000
request status: 584	10000
request status: 585	10000
request status: 586	10000
request status: 587	10000
request status: 588	10000
request status: 589	10000
request status: 590	10000
request status: 591	10000
request status: 592	10000
request status: 593	10000
request status: 594	10000
request status: 595	10000
request status: 596	10000
request status: 597	10000
request status: 598	10000
request status: 599	10000
request status: 600	10000
request status: 601	10000
request status: 602	10000
request status: 603	10000
request status: 604	10000
request status: 605	10000
request status: 606	10000
request status: 607	10000
request status: 608	10000
request status: 609	10000
request status: 610	10000
request status: 611	10000
request status: 612	10000
request status: 613	10000
request status: 614	10000
request status: 615	10000
request status: 616	10000
request status: 617	10000
request status: 618	10000
request status: 619	10000
request status: 620	10000
request status: 621	10000
request status: 622	10000
request status: 623	10000
request status: 624	

Três regras canônicas para atestações em federação:

1. **Issuer é parte da reputação.** Atestação assinada por auditor reconhecido pesa mais que auto-atestação. O verificador deve manter lista de issuers confiáveis.
2. **Toda atestação tem expiração.** Atestação de 2 anos atrás não prova nada sobre o agente de hoje.
3. **Atestações negativas existem.** "Agente X falhou auditoria em março/2026" é informação tão importante quanto sucesso. Frameworks sérios suportam ambas.

5. Reputação: cálculo, decay e ataques

Reputação numérica é o atalho que a federação usa para "vale a pena delegar a este agente?". Cálculo simplista (soma de sucessos / total) é fácil de manipular. Modelos mais robustos consideram:

Category	Percentage
Not at all	10%
Slightly	25%
Moderately	45%
Very much	20%



Sete vetores de ataque que sistemas de reputação ingênuos sofrem:

- **Sybil.** Atacante cria 10.000 agentes que se endossam mutuamente. Mitigação: penalizar redes com baixa diversidade de issuers.
- **Whitewashing.** Agente acumula reputação ruim, abandona identidade, recria nova limpa. Mitigação: idade de identidade pesa em score.
- **Boost coordenado.** Pequeno grupo dá ratings 5★ para amigos. Mitigação: peso por reputação do avaliador (PageRank-like).
- **Bait-and-switch.** Acumula reputação fazendo tarefas pequenas, abusa em uma tarefa grande. Mitigação: tarefas têm "peso" no histórico.
- **Reputation-as-a-service.** Mercado paralelo vende endorsements. Mitigação: detecção estatística + penalização severa quando comprovado.
- **Time decay errado.** Score atual reflete reputação de 2 anos atrás. Mitigação: half-life explícito (típico 90 dias para domínios voláteis).
- **Domínio cruzado.** Bom em coding, ruim em finance, score agregado esconde. Mitigação: scores por skill, não global.

Quem implementa reputação sem prever esses vetores entrega gameable system. Reputação real é tão complexa quanto a contabilidade — e exige o mesmo tipo de auditoria.

6. Capability tokens descendentes

Quando agente A delega para agente B (Volume V), B precisa agir **em nome de A** com escopo limitado. A primitiva canônica é o **capability token descendente** — análogo ao macaroon de Google, adaptado para agentes.





Propriedades canônicas:

- **Atenuação só, nunca elevação.** Token derivado pode reduzir escopo, nunca aumentar. Garantia criptográfica via assinatura encadeada.
- **Cadeia auditável.** `delegation_chain` registra quem autorizou o quê até o humano de origem.
- **Restrições contextuais.** IP, janela de tempo, contagem máxima.
- **Revogável.** Token pode ser invalidado antes do `exp` por lista de revogação (Capítulo 7).

A diferença prática vs OAuth puro: OAuth amarra escopo ao usuário; aqui o escopo viaja com a tarefa e atravessa N agentes mantendo trilha completa.

7. Revogação: o problema mais difícil

Identidade emitida é fácil. Identidade revogada de forma propagada é o problema sem solução elegante de toda a literatura.

Quatro mecanismos canônicos, todos com trade-offs:

CRL (Certificate Revocation List). Lista centralizada de IDs revogados. Verificadores consultam antes de aceitar. Funciona; é lento; não é cacheável corretamente.

OCSP-like (consulta em tempo real). "Esse ID ainda é válido?". Latência adicional por chamada; ponto único de falha do responder.

Short-lived credentials. Em vez de revogar, emitir credenciais curtas (1-4h) e simplesmente não re-emitir. Elegante; exige infraestrutura de renovação ativa.

Status List 2021 (W3C). Bitmap em URL pública; cada credencial referencia índice no bitmap; verificador faz GET. Funciona offline (cacheável); só precisa baixar bitmap uma vez por janela.

A combinação canônica para 2026: **short-lived credentials para uso diário + Status List para emergências.** Operadores que confiam apenas em CRL legacy descobrem na hora errada que a propagação demora horas.

Cenário pior: agente comprometido, token vazado, atacante usando em outro agente. O tempo entre detecção e revogação propagada é a janela de exposição. Empresas sérias medem essa janela como métrica de incident response.

8. Trust frameworks setoriais

Federação cross-org não acontece em vácuo. Acontece dentro de **trust frameworks setoriais** que definem regras de identidade, atestação e revogação para uma indústria.

Frameworks em consolidação em 2026:

- **Open Banking AI** — padrão para agentes financeiros, herda KYC/AML do banking tradicional.
- **HealthAgent Trust** — agentes em saúde, com requisitos HIPAA/LGPD específicos.
- **GovAI Roster** — agentes que operam para governo, com listas de conformidade.
- **EU AI Act Risk Registry** — agentes "high-risk" precisam de registro público obrigatório.

Trust framework define três coisas:

1. **Quem pode ser issuer.** Lista de auditores e autoridades aceitas.
2. **Que atestações são exigidas.** Mínimo de credenciais para operar.
3. **Como funciona revogação.** Mecanismo e SLA propagação.

Operar fora de framework setorial em domínio regulado é viável tecnicamente, **inviável legalmente.** Quem ignora isso colhe notificações regulatórias antes do produto-market fit.

9. Identidade do operador humano vs identidade do agente

A pergunta jurídica não-resolvida em 2026: quando o agente age, **quem agiu?** O agente, o operador, o desenvolvedor da skill, o provedor do modelo?

A doutrina que está prevalecendo (EU AI Act, NIST AI RMF) é **responsabilidade do operador**: a entidade que coloca o agente em operação para tarefas específicas é responsável final pelas consequências. Provedor de modelo tem responsabilidade limitada (defeito no modelo); desenvolvedor de skill tem responsabilidade pelo contrato declarado.

Isso desenha a arquitetura de logs que precisa existir:

- **Log assinado pelo operador** — registra que tarefa foi autorizada.
- **Log assinado pelo agente** — registra decisões e ações.
- **Log do modelo (no provedor)** — registra inferências executadas.
- **Log da skill (no skill provider)** — registra execuções.

Em incidente, a investigação cruza os quatro logs para estabelecer cadeia de causalidade. Sistema que não produz essa quádrupla é sistema não-auditável — e na prática, não-defensável em processo.

10. Manifesto: confiança sem rastro é credulidade

A indústria descobriu nos últimos anos que reputação numérica em e-commerce e gig economy é manipulável (avaliações falsas, bots). Sistemas agênticos estão prestes a importar todos esses problemas, com agravante: o agente mente em escala industrial.

Confiança real em federação agêntica não vem de: - "Confio porque o nome soa profissional." - "Confio porque tem rating 4.8★." - "Confio porque está na lista do registry X."

Vem de: - Identidade verificável criptograficamente. - Atestações de terceiros independentes, datadas e revogáveis. - Histórico de execução com proveniência rastreável. - Capacidade de revogar e penalizar em escala. - Framework setorial com responsabilidade legal definida.

Quem opera federação agêntica sem essa pilha está rezando, não arquitetando.

Tese final do volume: A camada de confiança agêntica não é um diferencial competitivo — é pré-condição para qualquer sistema multi-agente atravessar fronteira organizacional. Federação sem trust layer é o equivalente de uma bolsa de valores sem custódia: funciona até alguém perceber.

Checklist Canônico — Trust Layer

- [] Distinção operator_id / agent_id / model_id em toda audit trail.
- [] DIDs (did:web preferencial) publicados para todos os agentes.
- [] Agent Passport com signature do operator e validade definida.
- [] Atestações verificáveis de terceiros (auditoria, outcome, compliance).
- [] Reputação calculada por skill, com decay, e protegida contra Sybil.
- [] Capability tokens descendentes com atenuação criptográfica.
- [] Mecanismo de revogação testado (Status List + short-lived combo).
- [] Trust framework setorial mapeado e atendido.
- [] Logs quádruplos (operator/agent/model/skill) preservados.
- [] Plano de resposta a comprometimento de identidade documentado.

Glossário do Volume

- **DID** — Decentralized Identifier (W3C).
- **did:web** — método DID resolvido via HTTPS em domínio próprio.
- **VC** — Verifiable Credential, atestação assinada.
- **Agent Passport** — documento de identidade do agente, assinado pelo operator.
- **Capability token** — token de autoridade delegada com escopo limitado.
- **Atenuação** — propriedade de tokens derivados: só reduz escopo.
- **CRL** — Certificate Revocation List.
- **Trust framework** — conjunto setorial de regras de identidade/atestação.

Gancho para o Próximo Volume

Identidade prova quem é. Mas mesmo agentes confiáveis degradam — modelos mudam, dados driftam, prompts envelhecem. Como saber se a rede de agentes está melhor hoje que ontem? Como detectar regressão antes do cliente? Esse é o terreno do **Volume VIII — Evals, Observability e Telemetria Agêntica**.

